# Future Control Moves Memory: A Long-Horizon Moved-Bottleneck Diagnostic for Synthetic Agents

**Jawaun Brown** 2026-07-03

## Abstract

This paper asks a narrow question about finite neural agents: when one early state variable becomes future-critical for a delayed decision, does the agent's memory geometry become specifically sensitive to that variable, and does that sensitivity move when the critical variable moves?

We introduce the **long-horizon moved-bottleneck diagnostic**. Four early clue slots are matched for salience and frequency. One slot is selected as the future-critical bottleneck, moved across registered positions, and queried only after a long delay. The primary metric is not just final accuracy; it is the relative displacement of the final hidden state under single-slot bit flips. The registered gate requires the final-state sensitivity peak to follow the moved critical slot while remaining absent in a visible-control condition.

The result is positive across a ladder of increasingly agent-like synthetic regimes. A transformer on Modal `L4` solves the base delayed-memory task, survives longer horizons through sequence length 384, and preserves the moved bottleneck through closed-loop external-state handoff, repair after tool error, parsed structured calls, multi-field tool schemas, stochastic API failures, an 8-slot larger argument namespace, an alias-rich argument surface, parser-facing text arguments, fixed-length generated JSON strings, and autoregressively decoded JSON-like action strings. Across the confirmed Modal reports, the relevant bottleneck groups reach 1.000 closed-loop accuracy and positive moved-slot specificity, while visible controls keep specificity near zero.

The first prompt-level transfer produced a useful controlled strong negative at one hidden site: `Qwen/Qwen2.5-0.5B-Instruct` emits valid parser-scored JSON actions and passes the behavioral gates, but final-prompt hidden specificity is not confirmed. A follow-up localization sweep resolves that ambiguity. Across `Qwen/Qwen2.5-0.5B-Instruct`, `Qwen/Qwen2.5-1.5B-Instruct`, and `HuggingFaceTB/SmolLM2-1.7B-Instruct`, all behavior controls pass and 17 registered hidden sites pass. The strongest signal appears at final generated JSON tokens, with additional late/final prompt-token sites passing for Qwen 1.5B.

The allowed claim is deliberately modest: in this synthetic neural-agent setting, **future control relevance can move finite memory-state and tool-commitment sensitivity**. This is not a production-agent benchmark, production-agent benchmark or consciousness claim. Its value is as a cheap, reproducible diagnostic for whether an agent's internal state and generated action trajectory track the variables that will later control action.

## 1. Motivation

Many long-horizon agent evaluations ask whether an agent eventually gives the right answer. That is necessary but weak. A system can succeed by a shortcut, by storing everything uniformly, by using local query cues, or by relying on a privileged final token. The moved-bottleneck diagnostic tests a sharper property: whether the agent's internal memory metric follows the variable that will matter later.

This matters for three fields.

1. **Interpretability:** it gives a controlled way to ask where future-relevant information lives in a model's hidden state.
2. **Agent evaluation:** it tests delayed commitment and repair, not just answer selection at the final step.
3. **Safety and monitoring:** it suggests a cheap diagnostic for variables that silently become control bottlenecks in long-horizon systems.

The experiment is intentionally synthetic. That is a feature, not a defect, for this stage: the task isolates memory transport, critical-variable movement, and tool-interface hardening before introducing the confounds of natural language and real

APIs.

## 2. Diagnostic Design

Each episode contains matched early clues:

```
slot_0: bit b0
slot_1: bit b1
slot_2: bit b2
slot_3: bit b3
...
delayed query
```

In the `bottleneck` condition, the correct final action is the bit at one registered critical slot. The critical slot is moved across runs. In the `visible_control` condition, the final answer is visible at the query token and the early clues are matched distractors.

For each trained cell, the metric intervention flips one early clue bit at a time and measures the final hidden-state displacement. Slot densities are z-scored within model. The primary specificity statistic is:

```
z_density(critical_slot) − mean z_density(noncritical_slots)
```

The core gates are:

- behavior: bottleneck accuracy at least 0.90;
- metric transport: bottleneck memory-specificity CI lower bound above zero;
- rank: the critical slot ranks above chance among registered slots;
- visible-control null: visible-control specificity remains below 0.50.

The tool-interface regimes add gates for committed slot/value accuracy, schema validity, parsed action validity, repair behavior, stochastic failure handling, and no-op behavior after successful first calls.

## 3. Evidence Ladder

The experiment was advanced through a sequence of regimes. Each regime keeps the moved critical slot and visible-control null, then changes one surface of the agent's interaction with future-relevant information.

| Regime | What changed | Cells | Key signal | Report key |
|---|---|---|---|---|
| Base moved bottleneck | Delayed query over four matched clue slots | 64 | Bottleneck final accuracy 1.000; memory specificity +2.309; visible null | `base` |
| Horizon stress | Delay lengths 128, 256, 384 | 96 | All lengths preserve final accuracy 1.000 and moved-slot specificity | `horizon` |
| Closed-loop tool handoff | Model commits slot/value to recover external state | 32 | Tool bottleneck final accuracy 1.000; slot/value commitment 1.000 | `commit` |
| Repair bottleneck | First tool attempt receives an error; model must repair | 48 | Repair condition final accuracy 1.000; repair slot/value 1.000 | `repair` |

| | | | | |
|---|---|---|---|---|
| Structured tool call | One parsed JSON-like action token replaces direct heads | 48 | Direct and repair structured calls pass schema and parsed-slot gates | `structured` |
| Multifield schema | Separate opcode, slot, and value fields | 48 | Multifield direct/repair groups pass field, schema, and parsed-value gates | `multifield` |
| Stochastic tool failure | First-call success/ failure sampled per episode | 32 | Failed repair and success no-op both 1.000; failure rate 0.506 | `stochastic` |
| 8-slot stochastic | Larger argument namespace and longer sequence | 64 | 8-slot stochastic gates pass; memory specificity +3.023 | `8slot` |
| Alias argument surface | Three equivalent aliases per canonical slot | 32 | Alias parsed slot/value, failed repair, and success no-op all 1.000 | `alias` |
| Text argument surface | Parser-facing phrases such as `second clue` replace alias IDs | 32 | Text parsed slot/value, failed repair, and success no-op all 1.000 | `text` |
| Generated JSON surface | Model emits a fixed-length JSON-like token sequence | 32 | Generated sequence/ schema, parsed slot/ value, and repair gates all 1.000 | `generated_json` |
| Autoregressive JSON surface | JSON action is decoded token-by-token from commit state | 32 | Greedy decoded sequence/schema, parsed slot/value, and repair gates all 1.000 | `autoregressive_json` |
| Prompt JSON transfer | Pretrained open model emits parser-scored JSON text | 64 | Controls and behavior pass; hidden specificity CI crosses zero | `prompt_json_transfer` |
| Prompt JSON hidden localization | Multi-model, multi-layer, multi-token hidden-site grid | 192 behavior cells + 576 hidden rows | Controls and behavior pass for three models; 17 hidden sites pass | `prompt_json_hidden_localization` |

All confirmed sweeps used Modal `L4`, not H100/H200. The timeout-based conservative spend guard for the listed confirmed reports is under USD 160 in aggregate; individual recent passes used guards of USD 8.63 to USD 17.26. Actual runtime was much lower than the timeout budget, with the latest autoregressive JSON pass averaging 15.08 seconds per remote cell. The prompt-level transfer used one `L4` container for all 64 logical cells, with a conservative timeout guard of USD 1.08 and a 212.0 second remote runtime. The prompt-level hidden-localization sweep used three parallel `L4` model jobs, 192 behavior cells, 576 hidden rows, and a conservative timeout guard of USD 6.47.

Report keys map to committed summaries under `experiments/long_horizon_bottleneck/results/`: `base` = `modal_transformer_l4_8seed_2026_07_02.md`, `horizon` = `modal_transformer_l4_horizon_4seed_2026_07_02.md`, `commit` = `z_closed_loop_tool_commitment_l4_4seed_2026_07_02.md`, `repair` = `zz_tool_recovery_l4_4seed_2026_07_02.md`, `structured` = `zzz_structured_tool_call_l4_4seed_2026_07_03.md`, `multifield` = `zzzz_multifield_tool_schema_l4_4seed_2026_07_03.md`, `stochastic` = `zzzzz_stochastic_tool_failure_l4_4seed_2026_07_03.md`, `8slot` = `zzzzzz_stochastic_tool_failure_8slot_l4_4seed_2026_07_03.md`, and `alias` = `zzzzzzz_alias_argument_surface_l4_4seed_2026_07_03.md`, `text` = `zzzzzzzz_text_argument_surface_l4_4seed_2026_07_03.md`, `generated_json` =

`zzzzzzzzz_generated_json_surface_l4_4seed_2026_07_03.md`, and `autoregressive_json` = `zzzzzzzzzz_autoregressive_json_surface_l4_4seed_2026_07_03.md`, and `prompt_json_transfer` = `zzzzzzzzzzz_prompt_json_transfer_l4_4seed_2026_07_03.md`, and `prompt_json_hidden_localization` = `zzzzzzzzzzzzz_prompt_json_hidden_localization_l4_4seed_2026_07_03.md`.

## 4. Prompt-Level JSON Transfer and Hidden Localization

The autoregressive JSON pass completed the synthetic ladder: the trained model could decode parser-scored JSON-like action strings token by token under stochastic first-call failure and conditional repair. The next question was whether the same diagnostic transfers to a pretrained prompt-level model.

The prompt-level transfer used `Qwen/Qwen2.5-0.5B-Instruct` on one Modal `L4` container. The model saw natural-language memory records and had to emit one JSON object:

```
{"tool":"read_slot","slot":"second clue","value":0}
```

or:

```
{"tool":"noop"}
```

The confirmatory run used 64 logical cells: 4 conditions, 4 moved critical slots, and 4 seeds, with 8 episodes per cell. Format, visible-control, and short-horizon controls passed. The behavioral bottleneck also passed: closed-loop final accuracy was 0.977, first parsed slot/value were 0.984/0.977, failed-repair slot/value were 1.000/1.000, and success repair no-op was 1.000.

The full prompt-level gate nevertheless failed by the preregistered hidden metric. Critical-slot memory specificity was +0.695, but its 95% bootstrap interval was [-0.376, 2.080], so the lower bound was not above zero. The rank metric was above chance by mean rank percentile (0.594), but the primary specificity gate did not pass.

This is a controlled strong negative, not an interface failure. The model can solve the parser-scored prompt-level moved-bottleneck behavior, including stochastic repair, but this run does not show reliable hidden-state transport of the future-critical slot.

The hidden-localization follow-up asked whether that negative was specific to the measured site. It ran three cheap open models in parallel on Modal `L4`: `Qwen/Qwen2.5-0.5B-Instruct`, `Qwen/Qwen2.5-1.5B-Instruct`, and `HuggingFaceTB/SmolLM2-1.7B-Instruct`. The sweep preserved the same behavior conditions, then measured hidden specificity at `prompt_final`, `generated_first`, and `generated_final` token positions across early, mid, late, and final layer aliases.

This run was positive. All behavior controls passed for all three models. Behavioral bottleneck accuracy was 1.000 for SmolLM2 1.7B, 0.961 for Qwen 0.5B, and 0.938 for Qwen 1.5B. Seventeen registered hidden sites passed both the specificity and rank gates. The strongest sites were generated-final states: SmolLM2 1.7B generated-final/mid had specificity +228.301 with CI [129.438, 386.540]; Qwen 1.5B generated-final/early had +216.902 with CI [119.175, 332.137]; and Qwen 0.5B generated-final/early had +183.444 with CI [99.032, 279.601]. Qwen 1.5B also passed late/final prompt-final sites.

The interpretation is therefore narrower and stronger than the first prompt run: final behavior transfers, and hidden localization is present in the generated action trajectory. The original negative was a measurement-site failure, not evidence that prompt-level hidden geometry is absent.

## 5. Interpretation

The core result is a **metric transport** result. The future-critical variable does not merely determine the final label; it becomes the slot to which the agent's final hidden state and commitment heads are specifically sensitive. When the critical slot moves, the sensitivity peak moves with it. When the final answer is visible at query time, the early-slot peak

disappears.

The tool regimes add a second claim: the moved bottleneck survives when the agent has to externalize the critical variable through a tool interface. It must commit an argument, receive or fail to receive an external return, repair when needed, no-op when not needed, and still preserve the correct delayed decision.

This makes the diagnostic more relevant to long-horizon agents than to ordinary sequence classifiers. The bottleneck is not just "remember bit 2." It becomes "know which early variable must later be committed through an interface, recover it under feedback, and ignore matched distractors."

The prompt-level results sharpen the interpretation. Final behavior alone is not enough: a small open model can pass the prompt-level JSON behavior while one registered hidden site fails. But localization across token positions and layers recovers a positive signal, especially in generated action states. That split is the point of the diagnostic: it can distinguish interface behavior, prompt-state memory, and action-surface commitment.

## 6. Boundaries

The strongest honest statement is:

> Future control relevance can move finite memory and commitment sensitivity in a synthetic long-horizon neural agent, and that moved bottleneck survives progressively harder parsed tool-interface regimes.

The result does not establish:

- production API reliability;
- autonomous language-agent tool use;
- robustness across natural-language prompt families;
- multi-step planning in an open environment;
- human cognition or consciousness.

The latest prompt-level result uses a real pretrained tokenizer and generated JSON text, but it is still a compact harness, not an autonomous agent or real API environment.

## 7. Why This Is Valuable

The experiment's field value is not that it solves a benchmark. It gives a small, inspectable test for a pattern that current agent evaluations often miss: future relevance can reorganize internal memory geometry and action commitment.

A useful follow-on benchmark could ask, for a real agent or language model:

1. Which early facts become future-critical under a delayed objective?
2. Do internal representations or tool-call logits become specifically sensitive to those facts?
3. Does that specificity move when the objective moves?
4. Does it remain absent when the answer is visible later?
5. Does it survive tool failure, repair, and aliasing of argument names?

That is a sharper evaluation target than final-task success alone.

## 8. Remaining Work

The synthetic mechanism ladder is complete enough to write up and share, and the prompt-level hidden-localization replication is now positive. The next regimes answer different questions:

- **Prompt-level robustness:** rerun the frozen gate across prompt families, longer contexts, and API models to test whether the generated-token localization signal is robust rather than prompt-specific.
- **Causal hidden localization:** use activation patching or fixed-action counterfactuals to separate prompt-state

memory from action-token identity.

- **LLM-agent transfer:** add longer tool contexts, natural-language argument aliases, and API-style parser recovery around the prompt-level version.
- **Multi-step planning:** require two or more future commitments where the critical bottleneck changes after intermediate feedback.
- **Interpretability probes:** compare the hidden-state metric to attention, activation patching, or linear probes over the critical slot.

The most valuable immediate next step is to publish the diagnostic as a compact benchmark card: synthetic ladder positive, prompt-level behavior positive, and prompt-level hidden localization positive with a clear caveat that the strongest signal is in generated action states.

## 9. Reproducibility

Experiment code lives in `experiments/long_horizon_bottleneck/`.

Synthetic ladder runner:

```
doppler --scope /Users/jawaun/superoptimizers run -- \
    uvx --python 3.12 --from modal modal run \
    experiments/long_horizon_bottleneck/modal_stochastic_tool_failure_sweep.py \
    --seeds 4 --train-steps 900 --architectures transformer \
    --conditions autoregressive_json_bottleneck,autoregressive_json_visible_control \
    --critical-slots 0,1,2,3 \
    --aliases-per-slot 3 \
    --failure-probability 0.5 \
    --budget-usd 25 \
    --out artifacts/long_horizon_bottleneck/autoregressive_json_surface_l4.json
```

Prompt-level confirmatory runner:

```
doppler --scope /Users/jawaun/superoptimizers run -- \
    uvx --python 3.12 --from modal modal run \
    experiments/long_horizon_bottleneck/modal_prompt_json_transfer_sweep.py \
    --model-id Qwen/Qwen2.5-0.5B-Instruct \
    --seeds 4 \
    --episodes-per-cell 8 \
    --hidden-metric-episodes 2 \
    --critical-slots 0,1,2,3 \
    --budget-usd 25 \
    --base-seed 20260800 \
    --out artifacts/long_horizon_bottleneck/prompt_json_transfer_l4.json
```

Prompt-level hidden-localization runner:

```
doppler --scope /Users/jawaun/superoptimizers run -- \
    uvx --python 3.12 --from modal modal run \
    experiments/long_horizon_bottleneck/modal_prompt_json_hidden_localization_sweep.py \
    --models Qwen/Qwen2.5-0.5B-Instruct,Qwen/Qwen2.5-1.5B-Instruct,HuggingFaceTB/SmolLM2-1.7B-
Instruct \
    --seeds 4 \
    --episodes-per-cell 8 \
    --hidden-metric-episodes 2 \
    --critical-slots 0,1,2,3 \
    --hidden-positions prompt_final,generated_first,generated_final \
    --hidden-layers early,mid,late,final \
    --budget-usd 25 \
```

```
    --base-seed 20260850 \
    --out artifacts/long_horizon_bottleneck/prompt_json_hidden_localization_l4.json
```

Local verification for the code paths:

```
uvx ruff check experiments/long_horizon_bottleneck tests/test_long_horizon_bottleneck.py
uvx ty check experiments/long_horizon_bottleneck tests/test_long_horizon_bottleneck.py
python -m compileall -q experiments/long_horizon_bottleneck tests/test_long_horizon_bottleneck.py
python -m pytest tests/test_long_horizon_bottleneck.py -q
```

The raw Modal artifacts are kept under gitignored `artifacts/`; committed result reports in `experiments/long_horizon_bottleneck/results/` summarize every run.